Bellman-ford + SPFA

人员

王恩泽、郭栩睿、于家瑞、于霄龙、王恩泽 到课

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3838

比赛概况 题目列表 选择题列表 提交记录 实时榜单 选择题排行榜 编辑比赛

王向东老师周六十点半C++图论综合练习

刷新

#	用户名	姓名	编程分	时间	A	B	C
1	guoxurui	郭栩睿	300	10889	100	100	100
2	wangenze	王恩泽	300	10963	100	100	100
3	hongchendong	洪晨栋	200	1846	100		100
4	yujiarui	于家瑞	155	45	100	55	
5	hongchenqi	洪晨棋	100	47	100		

本周作业

https://cppoj.kids123code.com/contest/4028 (课上讲了 A ~ B 这些题, 课后作业是 C 题)

课堂表现

今天讲了 Bellman-ford 和 SPFA 两个算法, 同学们上课听讲都很认真, 吸收的也都很不错。

课堂内容

【模板】单源最短路径（弱化版）

bellman-ford 模板, 会超时, 70 分代码

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e4 + 5;
const int inf = 0x3f3f3f3f3f3f3f3f;
struct Edge {
  int from, to, value;
};
int dis[maxn];

signed main()
{
```

```
  int n, m, id; cin >> n >> m >> id;
  vector<Edge> edges;
  while (m -- ) {
    int a, b, c; cin >> a >> b >> c; edges.push_back({a,b,c});
  }

  memset(dis, 0x3f, sizeof(dis)); dis[id] = 0;
  for (int i = 1; i <= n-1; ++i) {
    for (Edge it : edges) dis[it.to] = min(dis[it.to], dis[it.from]+it.value);
  }

  int limit = (1LL<<31) - 1;
  for (int i = 1; i <= n; ++i) {
    if (dis[i] == inf) cout << limit << " ";
    else cout << dis[i] << " ";
  }
  cout << endl;
  return 0;
}
```

SPFA 模板

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e4 + 5;
const int inf = 0x3f3f3f3f3f3f3f3f;
const int limit = (1LL<<31) - 1;
struct node {
  int to, value;
};
vector<node> vec[maxn];
int dis[maxn];
bool st[maxn];

signed main()
{
  int n, m, id; cin >> n >> m >> id;
  while (m -- ) {
    int a, b, c; cin >> a >> b >> c; vec[a].push_back({b,c});
  }

  queue<int> q;
  memset(dis, 0x3f, sizeof(dis));
  q.push(id); dis[id] = 0; st[id] = true;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    st[u] = false;

    for (node it : vec[u]) {
```

```
      if (dis[it.to] > dis[u]+it.value) {
        dis[it.to] = dis[u]+it.value;
        if (!st[it.to]) q.push(it.to), st[it.to] = true;
      }
    }
  }

  for (int i = 1; i <= n; ++i) {
    if (dis[i] == inf) cout << limit << " ";
    else cout << dis[i] << " ";
  }
  cout << endl;
  return 0;
}
```

【模板】负环

Bellman-ford 判负环

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e3 + 5;
const int inf = 0x3f3f3f3f;
struct Edge {
  int from, to, value;
};
int dis[maxn], f[maxn];
bool st[maxn];

void solve() {
  vector<Edge> edges;
  int n, m; cin >> n >> m;
  while (m -- ) {
    int a, b, c; cin >> a >> b >> c;
    edges.push_back({a,b,c});
    if (c >= 0) edges.push_back({b,a,c});
  }

  memset(dis, 0x3f, sizeof(dis)); dis[1] = 0;
  memset(st, false, sizeof(st)); st[1] = true;
  for (int i = 1; i <= n; ++i) {
    for (Edge it : edges) {
      int from = it.from, to = it.to, value = it.value;
      dis[to] = min(dis[to], dis[from]+value);
      st[to] |= st[from];
    }
    if (i == n-1) {
      for (int j = 1; j <= n; ++j) f[j] = dis[j];
    }
```

```
  }

  for (int i = 1; i <= n; ++i) {
    if (st[i] && f[i] != dis[i]) { cout << "YES" << endl; return; }
  }
  cout << "NO" << endl;
}

int main()
{
  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

SPFA 判负环

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e3 + 5;
const int inf = 0x3f3f3f3f;
struct node {
  int to, value;
};
vector<node> vec[maxn];
int dis[maxn], cnt[maxn];
bool st[maxn];

void solve() {
  int n, m; cin >> n >> m;

  for (int i = 1; i <= n; i ++ ) vec[i].clear();
  memset(dis, 0x3f, sizeof(dis));
  memset(cnt, 0, sizeof(cnt));
  memset(st, false, sizeof(st));

  while (m -- ) {
    int a, b, c; cin >> a >> b >> c; vec[a].push_back({b,c});
    if (c >= 0) vec[b].push_back({a,c});
  }

  queue<int> q;
  q.push(1); dis[1] = 0; cnt[1] = 1; st[1] = true;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    st[u] = false;

    for (node it : vec[u]) {
      if (dis[it.to] > dis[u]+it.value) {
        dis[it.to] = dis[u]+it.value; cnt[it.to]++;
```

```
          if (cnt[it.to] >= n) { cout << "YES" << endl; return; }
          if (!st[it.to]) q.push(it.to), st[it.to] = true;
        }
      }
    }

    cout << "NO" << endl;
  }

  int main()
  {
    int T; cin >> T;
    while (T -- ) solve();
    return 0;
  }
```